

Edge Intelligence – Log Data Analytics

Dhayanidhi R S

25MML0025

TASK: To find the outliers in log data for e-commerce use case.

DATASET:



ecommerce_logs.json

👉 Double click to see the

CODE:

```
import pandas as pd

json_file="ecommerce_logs.json"
# Read JSON
df = pd.read_json(json_file)

# Save as CSV
df.to_csv("ecommerce_logs.csv", index=False)

print("✅ Converted successfully!")
```

[1]

... ✅ Converted successfully!

```
df.isnull().sum()
```

```
log_id          0
timestamp       0
user_id         0
session_id      0
service         0
event_type      0
product_id      0
category        0
price           0
quantity        0
payment_method  0
payment_status  33242
response_time_ms 0
status_code     0
region          0
location        0
device          0
ip_address      0
is_fraud        0
cart_abandonment_spike 0
dtype: int64
```

```
# Step 1: Not applicable for non-purchase
df["payment_status"] = df.apply(
    lambda x: "not_applicable" if x["event_type"] != "purchase" else x["payment_status"],
    axis=1
)

# Step 2: Fill remaining missing
df["payment_status"] = df["payment_status"].fillna("unknown")
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df["timestamp"] = pd.to_datetime(df["timestamp"])

df["month"] = df["timestamp"].dt.month
```

```
▷ # Remove extreme values
df = df[df["price"] < df["price"].quantile(0.99)]
df = df[df["quantity"] < df["quantity"].quantile(0.99)]
```

[11]

```
▷ df.columns
```

[17]

```
... Index(['log_id', 'timestamp', 'user_id', 'session_id', 'service', 'event_type',
         'product_id', 'category', 'price', 'quantity', 'payment_method',
         'payment_status', 'response_time_ms', 'status_code', 'region',
         'location', 'device', 'ip_address', 'is_fraud',
         'cart_abandonment_spike', 'hour', 'day', 'month'],
         dtype='object')
```

```
▷ df["cart_dropoff"] = (
    (df["event_type"] == "add_to_cart") &
    (df["payment_status"] != "success")
).astype(int)
```

[18]

```
df["location_switch"] = df.groupby("user_id")["location"].transform("nunique")
```

[19]

```
df["region_change"] = (df["location_switch"] > 2).astype(int)
```

[20]

```
▷ df["is_fraud"].value_counts()
```

[4]

```
is_fraud
0      38753
1       780
Name: count, dtype: int64
```

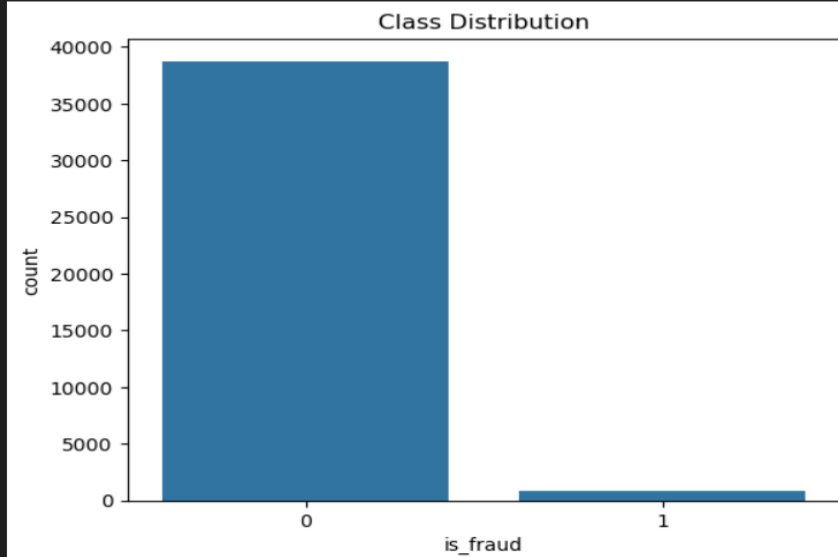
```
▷ df["is_fraud"].value_counts(normalize=True) * 100
```

[5]

```
is_fraud
0      98.026965
1       1.973035
Name: proportion, dtype: float64
```

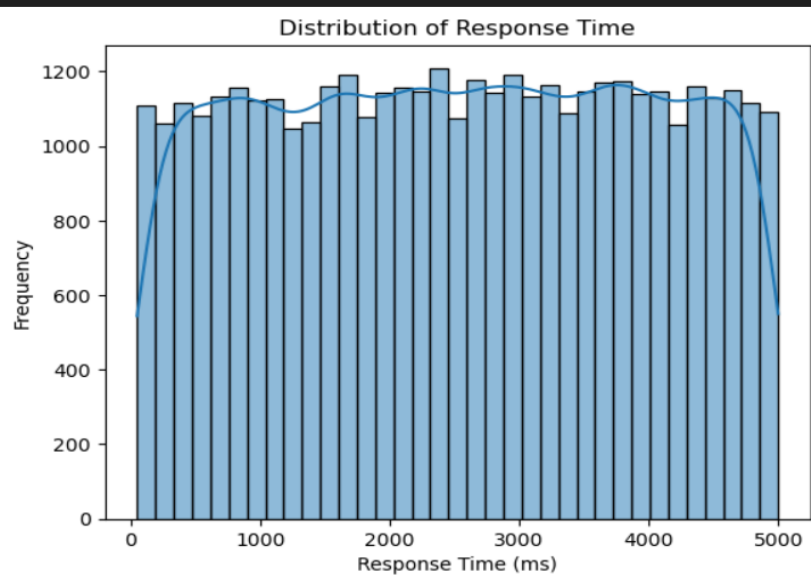
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x="is_fraud", data=df)
plt.title("Class Distribution")
plt.show()
```



```
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df["response_time_ms"], kde=True)
plt.title("Distribution of Response Time")
plt.xlabel("Response Time (ms)")
plt.ylabel("Frequency")
plt.show()
```



```

from sklearn.utils import resample

df_major = df[df.is_fraud == 0]
df_minor = df[df.is_fraud == 1]

df_major_down = resample(df_major,
                          replace=False,
                          n_samples=len(df_minor),
                          random_state=42)

df_balanced = pd.concat([df_major_down, df_minor])

```

```

from sklearn.ensemble import IsolationForest

cols = ["price", "quantity", "response_time_ms"]

iso = IsolationForest(contamination=0.02, random_state=42)

df["outlier_flag"] = iso.fit_predict(df[cols])

# -1 = outlier, 1 = normal
df["outlier_flag"] = df["outlier_flag"].map({1: 0, -1: 1})

```

```
df["outlier_flag"].value_counts()
```

```

outlier_flag
0    38742
1      791
Name: count, dtype: int64

```

```

import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

plt.scatter(df["price"], df["response_time_ms"],
            c=df["outlier_flag"], cmap="coolwarm", alpha=0.6)

plt.xlabel("Price")
plt.ylabel("Response Time")
plt.title("Outliers Detection using Isolation Forest")

plt.show()

```

